Introduction to Computer Science and Programming

Dynamic allocation and Dynamic arrays

Lecture 16, March 30, 2026

Filippo Ricca

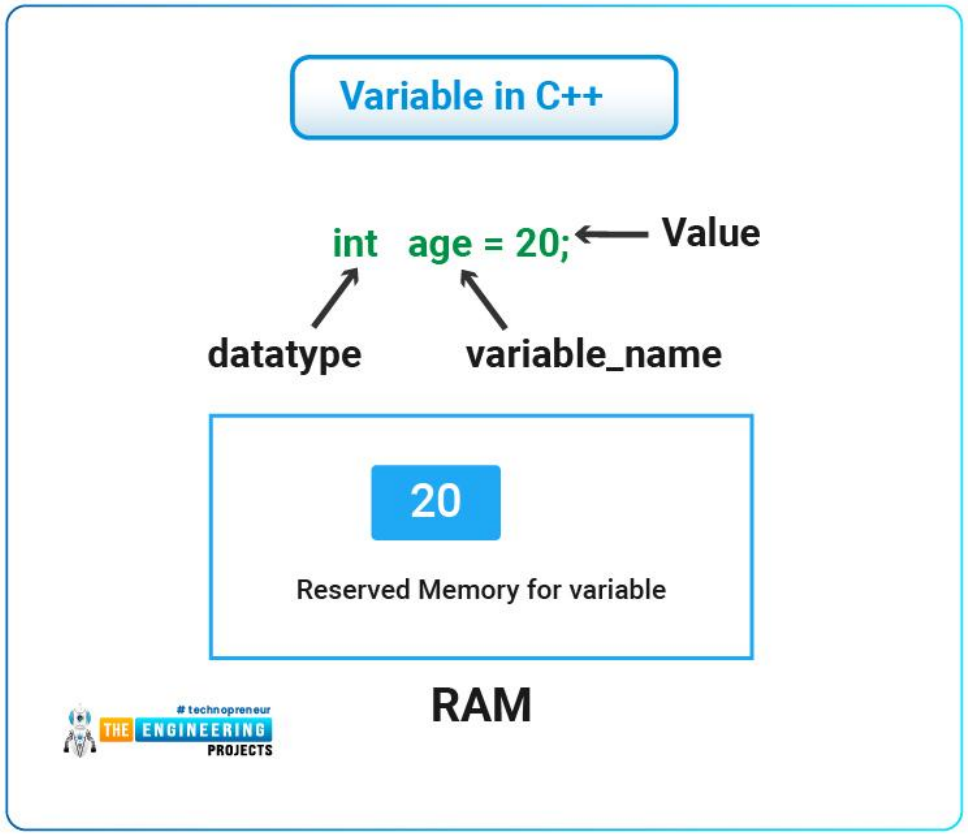
DIBRIS, University of Genova

Slides based on material provided by the Polytechnic University of Turin (C++ course) and IP course (University of Genova)

Dynamic allocation

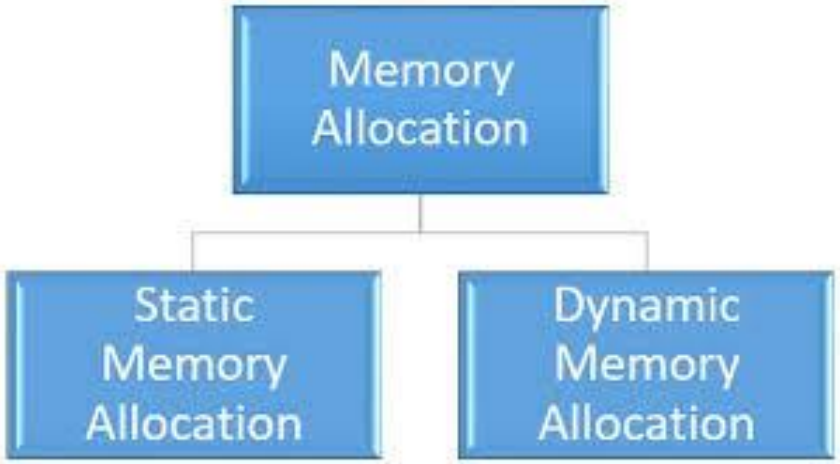
What Is Memory Allocation?

- Memory allocation is the **process of reserving memory for variables and data**
- It allows a program to store values, objects, and arrays in memory
- In C++, memory can be allocated in two main ways:
  - **Static allocation**: memory size is fixed in advance (compile time)
  - **Dynamic allocation**: memory is requested at runtime as needed
- Proper memory allocation is important for:
  - storing data efficiently
  - avoiding waste of memory
  - handling variable-size data structures



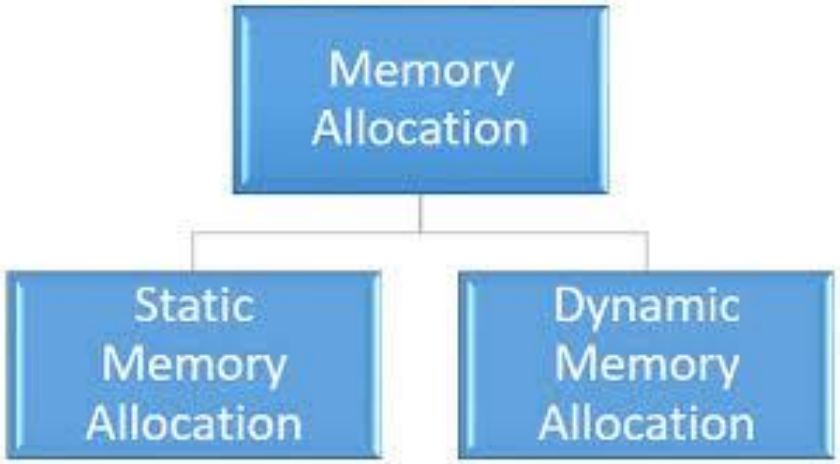
Static allocation

- **Static allocation** refers to the process of reserving memory at **compile time**, before the program starts executing
- The memory required for data structures is determined during the compilation, and **it remains fixed** for the duration of the execution
- This means that the size and number of these allocated elements **cannot change dynamically during runtime**
- In contrast, **dynamic allocation** would allow you to allocate memory during runtime



Static allocation

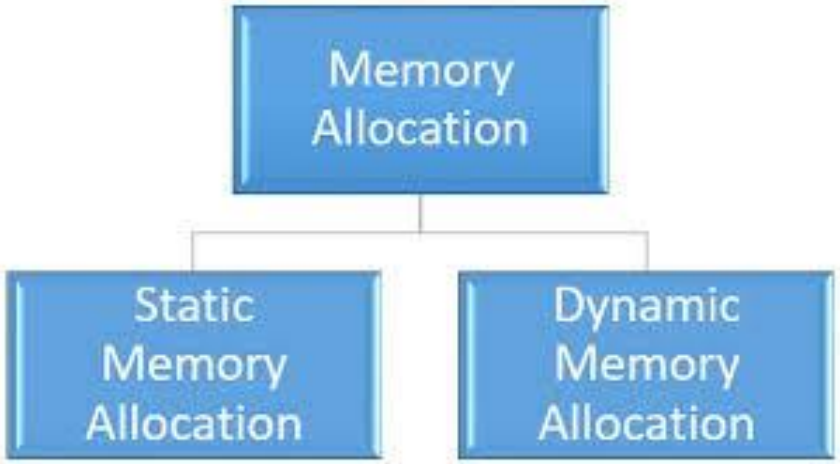
- **Static allocation** refers to the process of reserving memory at **compile time**, before the program starts executing
- The memory required for data structures is determined during the compilation, and **it remains fixed** for the duration of the execution
- This means that the size and number of these allocated elements **cannot change dynamically during runtime**
- In contrast, **dynamic allocation** would allow you to allocate memory during runtime



- Memory allocated at compile time
- Memory allocated is fixed
- Size cannot change

Static allocation

- **Static allocation** refers to the process of reserving memory at **compile time**, before the program starts executing
- The memory required for data structures is determined during the compilation, and **it remains fixed** for the duration of the execution
- This means that the size and number of these allocated elements **cannot change dynamically during runtime**
- In contrast, **dynamic allocation** would allow you to allocate memory during runtime



- Memory is reserved at compile time
- Memory allocated is fixed
- Size cannot change

- Memory is reserved at run time
- Memory allocated is not fixed
- Size can change

Static allocation example

```
int main() {
    int numbers[5] = {10, 20, 30, 40, 50};  // static allocation

    for (int i = 0; i < 5; i++) {
        cout << numbers[i] << " ";
    }

    return 0;
}
```

- ‘numbers’ is an array of fixed size 5 (**static** array)
- Its size is known before execution (**compile time**)
- The size of the array can not change during execution (**fixed**)
- Memory is allocated automatically (the compiler allocates it)

Cases where static allocation is fine

- Static arrays are good when the number of elements is **fixed and known in advance**
  - **We know the size and it won't change!!!**
- Days of the week

```
string days[7] = {"Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"};
```

- Months of the year

```
string months[12] = {"Jan", "Feb", "Mar", "Apr", "May", "Jun",
                     "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"};
```

- A small board with fixed size

```
char ticTacToe[3][3];
```

Problems with static allocation

- The main problem with **static allocation** is that the size is **fixed in advance**. This makes it simple, but also **inflexible**
  - With static allocation, you usually need to know the size in advance
  - If you allocate **more space** than you really need, some memory is **unused**
  - If you allocate **too little space**, you **cannot store more data** safely
  - Large statically allocated arrays inside a function may use too much stack memory
    - Overflow the stack
    - Crash!

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;

    // int arr[n];   // not standard C++
    return 0;
}
```

E.g., we need to store students in an array, but we do not know the number in advance

Motivation (static vs. dynamic)

- In many practical cases, we need to manipulate **dynamic structures**
  - e.g., when we do not know in advance the size that a given structure may reach
    - E.g., Number of students entered by the user
- The most effective choice is to allow the **program to dynamically allocate the necessary memory**

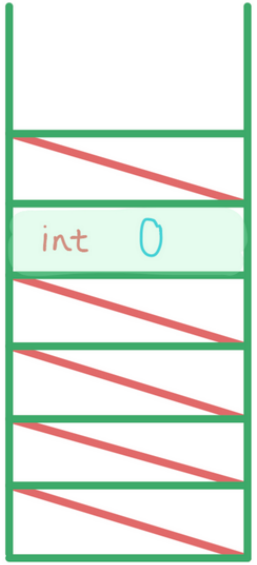
- Two options:
  - **Using the command 'new'**
    - This means the programmer **manually requests memory** at runtime
    - E.g., **Dynamic arrays**
  - **Automatically**
    - This usually means memory is managed **automatically by the language or by objects**
    - E.g., **std::vector**

Pointers and dynamic allocation

- The use of pointers so far (pointing to statically allocated variables) is not very interesting …
- However, pointers are essential for **dynamic memory management**
- Without a pointer, you would have **no way to refer to that memory after allocation**
- Pointers let the program keep track of where that memory is

Pointers and dynamic allocation

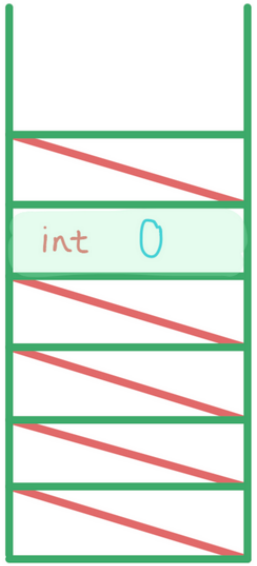
- The use of pointers so far (pointing to statically allocated variables) is not very interesting …
- However, pointers are essential for **dynamic memory management**
- Without a pointer, you would have **no way to refer to that memory after allocation**
- Pointers let the program keep track of where that memory is



We allocate a cell of memory

Pointers and dynamic allocation

- The use of pointers so far (pointing to statically allocated variables) is not very interesting …
- However, pointers are essential for **dynamic memory management**
- Without a pointer, you would have **no way to refer to that memory after allocation**
- Pointers let the program keep track of where that memory is



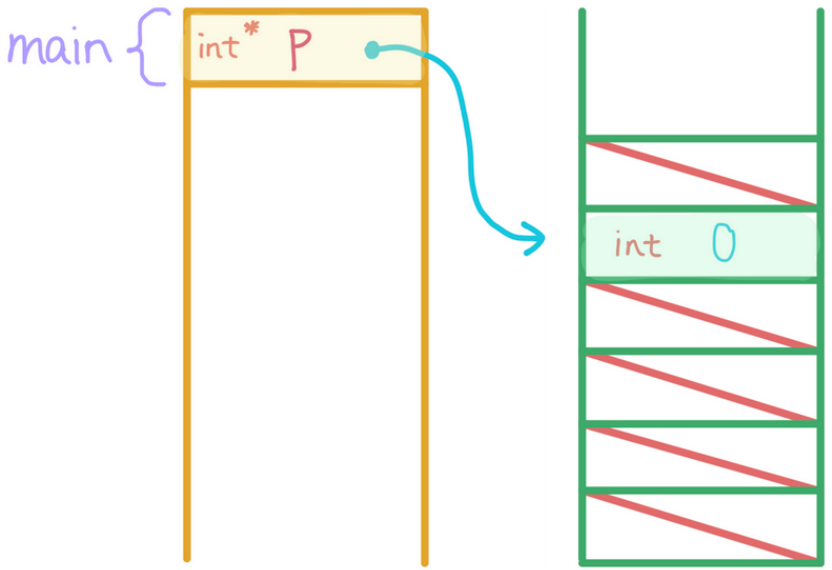
We allocate a cell of memory

How can we refer to it?

Pointers and dynamic allocation

- The use of pointers so far (pointing to statically allocated variables) is not very interesting …
- However, pointers are essential for **dynamic memory management**
- Without a pointer, you would have **no way to refer to that memory after allocation**
- Pointers let the program keep track of where that memory is

Using a pointer!!!



We allocate a cell of memory

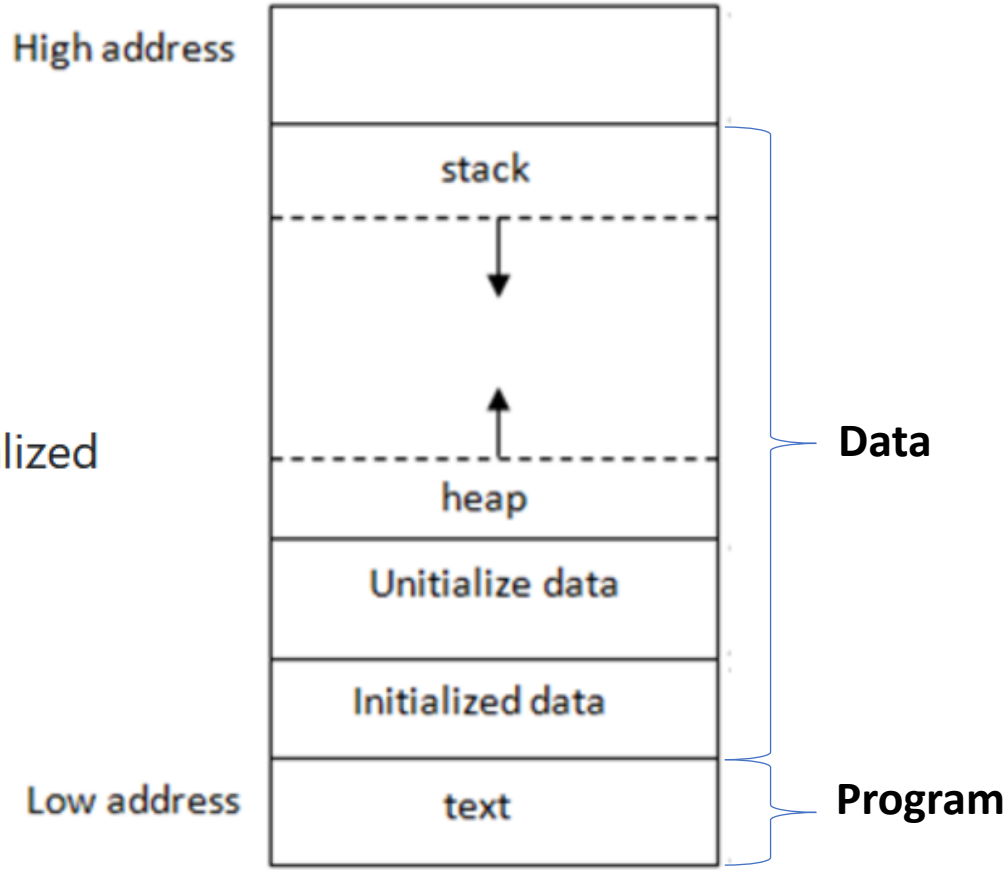
How can we refer to it?

Already introduced in Lecture 12 about Functions

Memory layout of C++ program

- When it is running, a C++ program has its own **memory layout**, separated from other programs

- `stack`: stores local variables
- `heap`: dynamic memory for programmer to allocate
- `data`: stores global variables, separated into initialized and uninitialized
- `text`: stores the code being executed



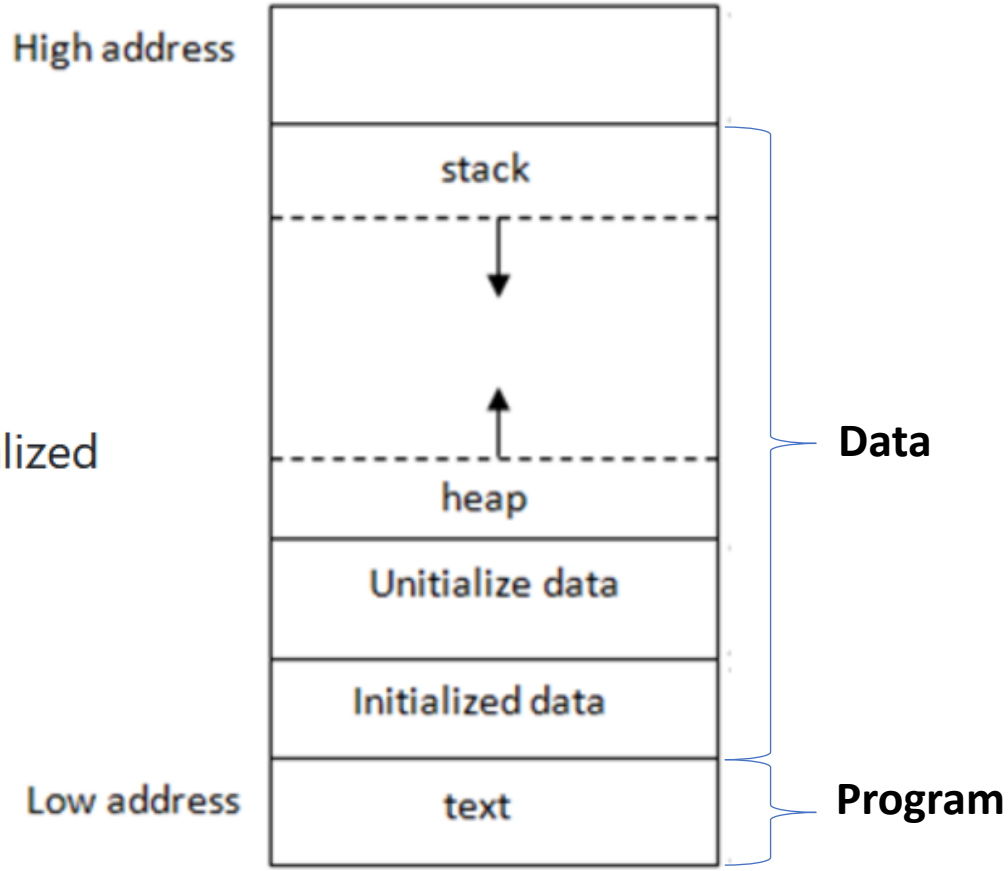
https://courses.grainger.illinois.edu/cs225/sp2023/resources/stack-heap/

Already introduced in Lecture 12 about Functions

Memory layout of C++ program

- When it is running, a C++ program has its own **memory layout**, separated from other programs

- `stack`: stores local variables Static allocation
- `heap`: dynamic memory for programmer to allocate
- `data`: stores global variables, separated into initialized and uninitialized
- `text`: stores the code being executed



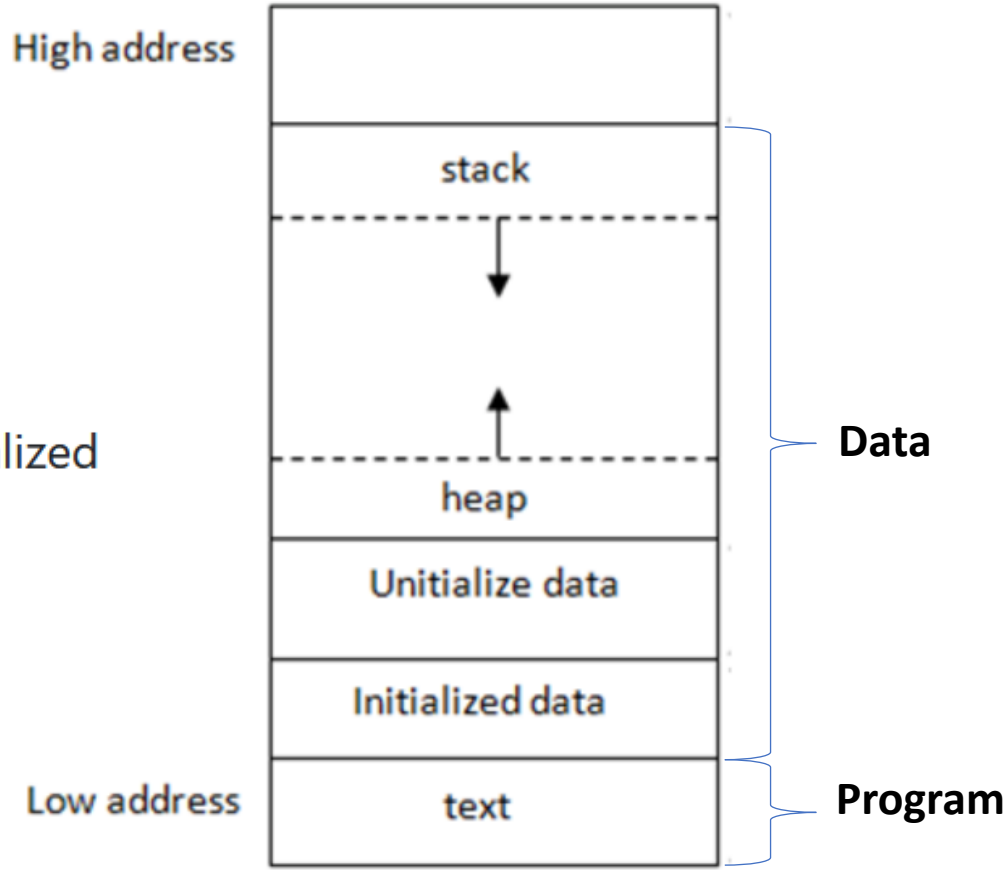
https://courses.grainger.illinois.edu/cs225/sp2023/resources/stack-heap/

Already introduced in Lecture 12 about Functions

Memory layout of C++ program

- When it is running, a C++ program has its own **memory layout**, separated from other programs

- `stack`: stores local variables Static allocation
- `heap`: dynamic memory for programmer to allocate
- `data`: stores global variables, separated into initialized and uninitialized
- `text`: stores the code being executed



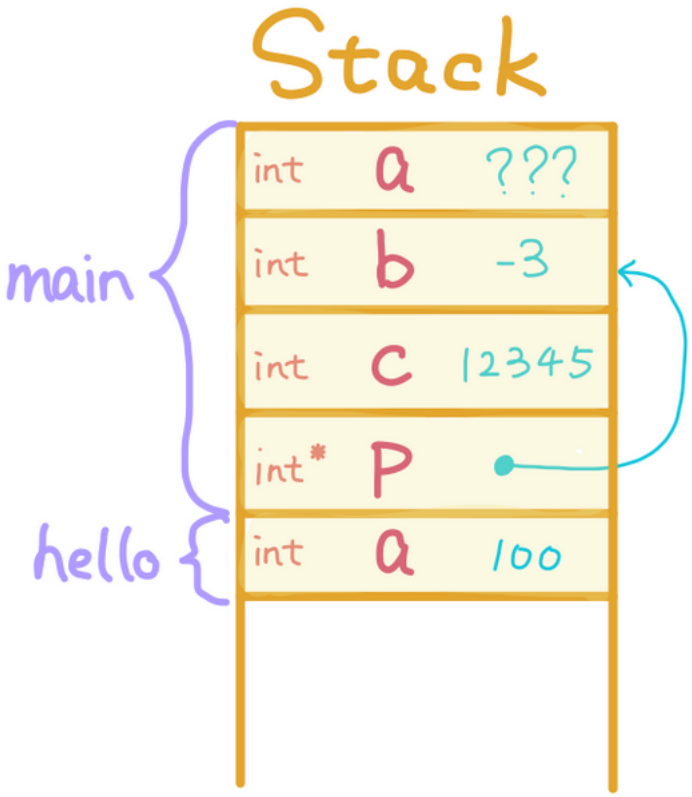
Dynamic structures are located in the heap!

https://courses.grainger.illinois.edu/cs225/sp2023/resources/stack-heap/

Already introduced in Lecture 12 about Functions

Stack

- Every **time a function is called, the machine allocates some stack memory** for it
  - Activation record
- When a **new local variables is declared, more stack memory is allocated** for that function to store the variable
- Such allocations make the stack grow downwards
- **The allocation and deallocation for stack memory is automatically done**



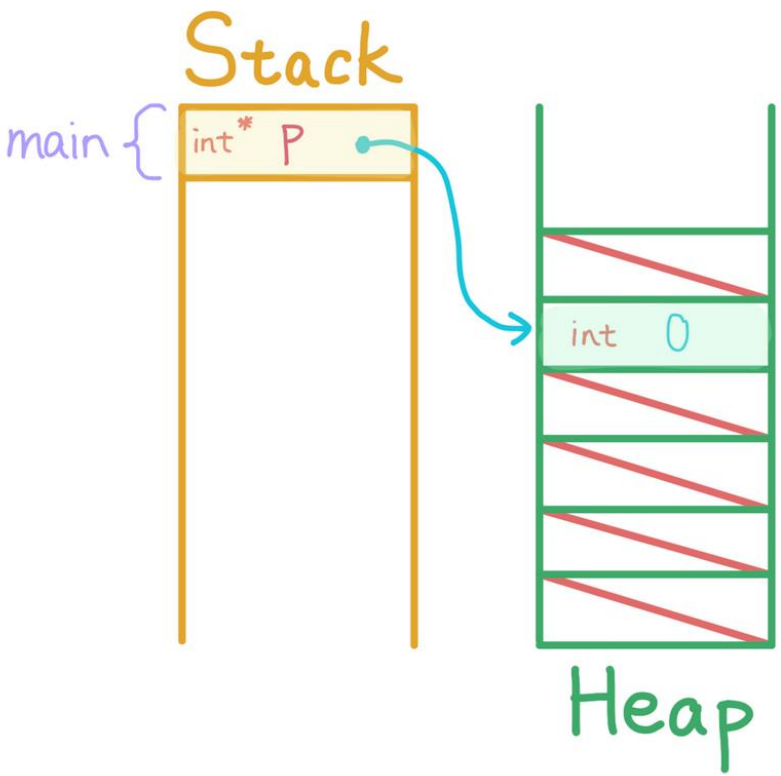

```
int hello() {
  int a = 100;
  return a;
}
int main() {
  int a;
  int b = -3;
  int c = 12345;
  int *p = &b;
  int d = hello();
  return 0;
}
```

The stack grows and shrinks automatically during the execution of the program

Heap

- Unlike stack memory, heap memory is **allocated explicitly by programmers,** and it won't be deallocated until it is explicitly freed

- To allocate heap memory in C++, use the keyword '**new**' followed of what you want to allocate
  - In our example **int**

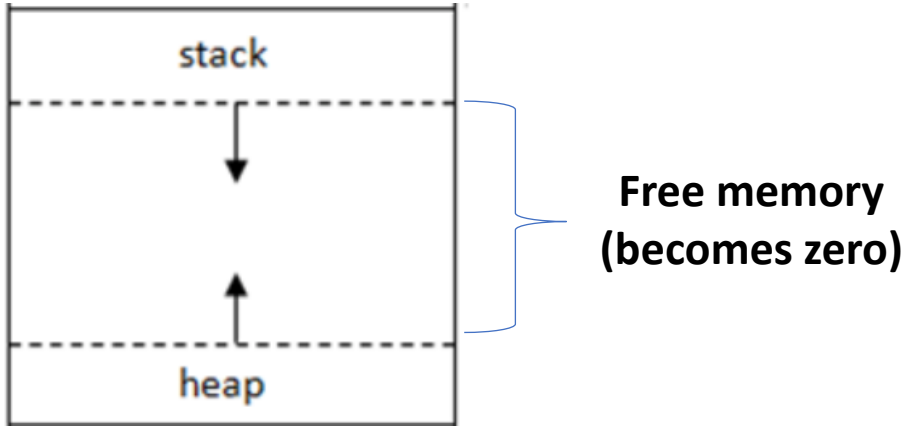



```
int main() {
  int *p = new int;
  Cube *c1 = new Cube();
  Cube *c2 = c1;
  c2->setLength( 10 );
  return 0;
}
```

P is a pointer to a memory's cell

What Happens When the Stack and Heap Collide

- **If the stack grows downward** (due to deeper function calls, large local variables, or recursion), **and the heap grows upward** (due to dynamic memory allocation), **they can eventually collide** if the program uses too much memory in either region



- This collision could cause a **segmentation fault**, a type of error in which the program **tries to access memory it isn't allowed to**, typically resulting in the **program crashing**

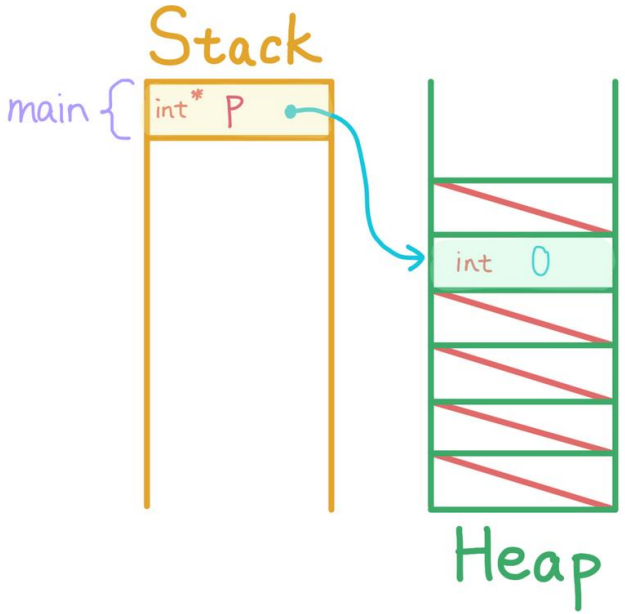
```
[01:18:40] Using gulpfile ~/Sites/sage-test/gulpfile.js
[01:18:40] Starting 'clean'...
[01:18:40] Finished 'clean' after 3.44 ms
[01:18:40] Starting 'default'...
[01:18:40] Starting 'build'...
[01:18:40] Starting 'wiredep'...
[01:18:40] Finished 'default' after 110 ms
[01:18:40] Finished 'wiredep' after 139 ms
[01:18:40] Starting 'styles'...
Segmentation fault: 11
Daenerys:sage-test Michael$
```

Heap manager

- I request the **heap manager** to allocate memory necessary to store a variable of a certain type
- The manager allocates the required memory on the heap and **returns the address**
- In C++, this operation is performed using the **new command**:

```
T *p;
p = new T;
```

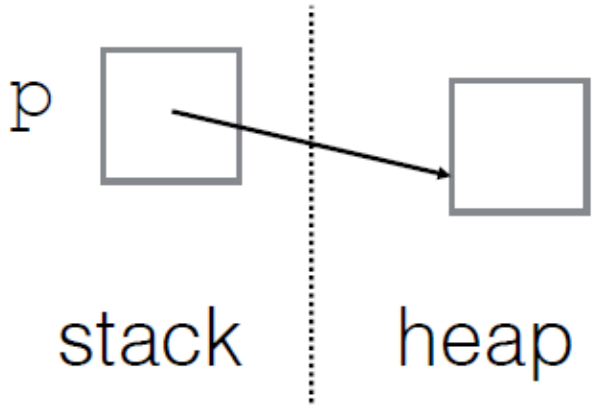
- This instruction **allocates a variable of type T** on the heap and stores its address in p




```
int main() {
  int *p = new int;
  Cube *c1 = new Cube();
  Cube *c2 = c1;
  c2->setLength( 10 );
  return 0;
}
```

Dynamic allocation (visual representation)

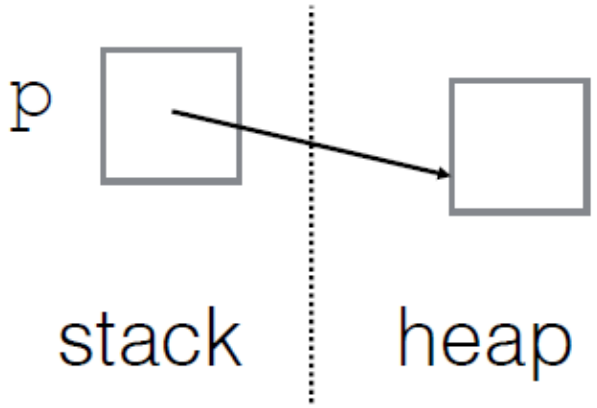
- T *p;
- `p = new T;`



The pointer itself is allocated on the stack, while the memory allocated with new is stored on the heap

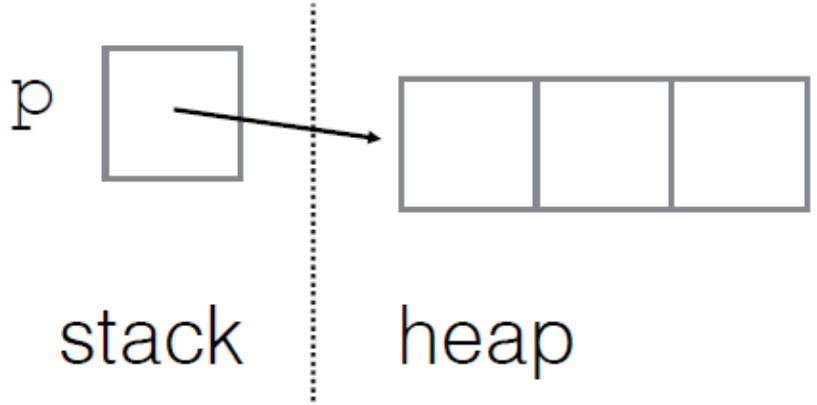
Dynamic allocation (visual representation)

- T *p;
- `p = new T;`



The pointer itself is allocated on the stack, while the memory allocated with new is stored on the heap

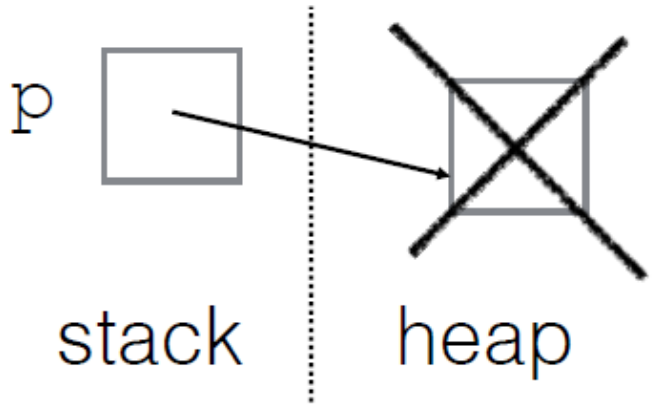
- `int size = 3;`
  `p = new T[size];`



Dynamic de-allocation

Returns a portion of memory to the heap that is no longer needed

- This operation invalidates access to that memory portion and makes it available to the heap manager for future use



Dynamic de-allocation

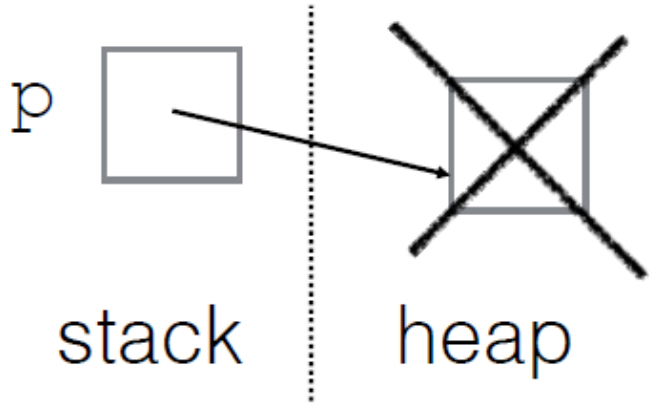
Returns a portion of memory to the heap that is no longer needed

- This operation invalidates access to that memory portion and makes it available to the heap manager for future use
- In C++, this operation is performed using the delete command:

```
delete p;
```

- If we have a structure with multiple elements:

```
delete [] p;
```



Dynamic de-allocation

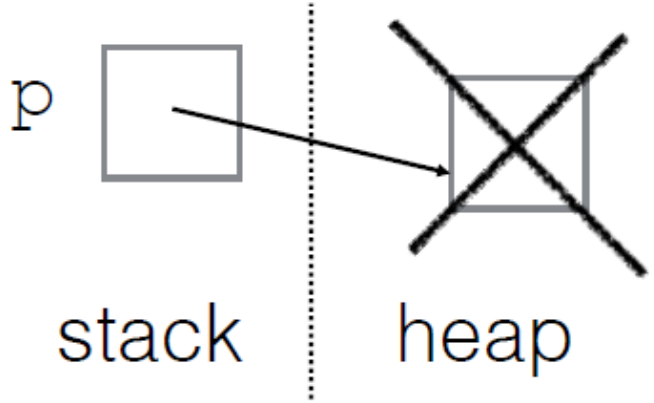
Returns a portion of memory to the heap that is no longer needed

- This operation invalidates access to that memory portion and makes it available to the heap manager for future use
- In C++, this operation is performed using the delete command:

```
delete p;
```

- If we have a structure with multiple elements:

```
delete [] p;
```



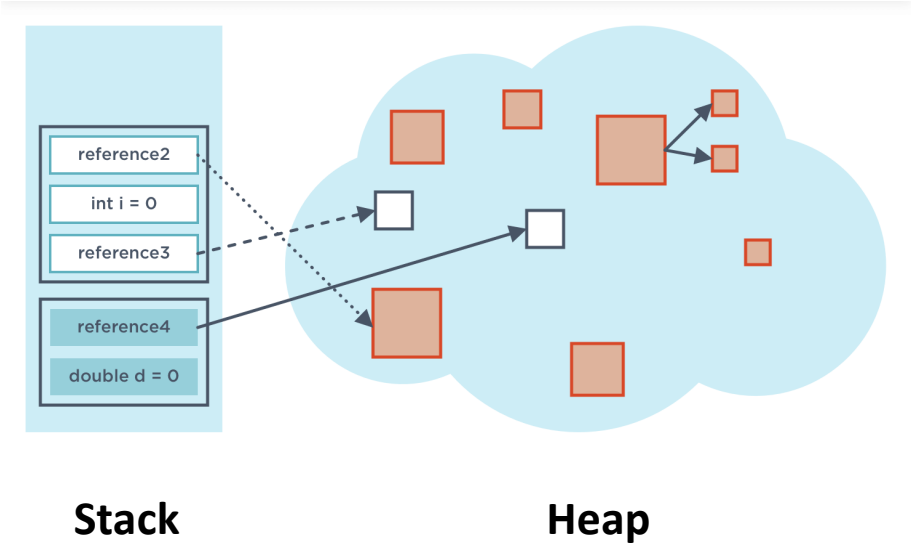
After delete, the pointer still contains the old address, but that address is no longer safe to use

Problems with Dynamic allocation

- Pay attention to:
  - **Dangling pointers**
    - A **dangling pointer** happens when a pointer still points to memory that has already been deallocated

Problems with Dynamic allocation

- Pay attention to:
  - **Dangling pointers**
    - A **dangling pointer** happens when a pointer still points to memory that has already been deallocated
  - **Memory leak**
    - A **memory leak** happens when a program allocates memory dynamically, but **never releases it**



Dangling Pointers problems

- Dangling pointers cause **undefined behavior** because they point to memory that is **no longer valid**

- Main problems:
  - **crashes**
    the **program may crash** when trying to access freed memory

  - **wrong values**
    the pointer may **read garbage** or unpredictable data

ptr

Dangling Pointers problems

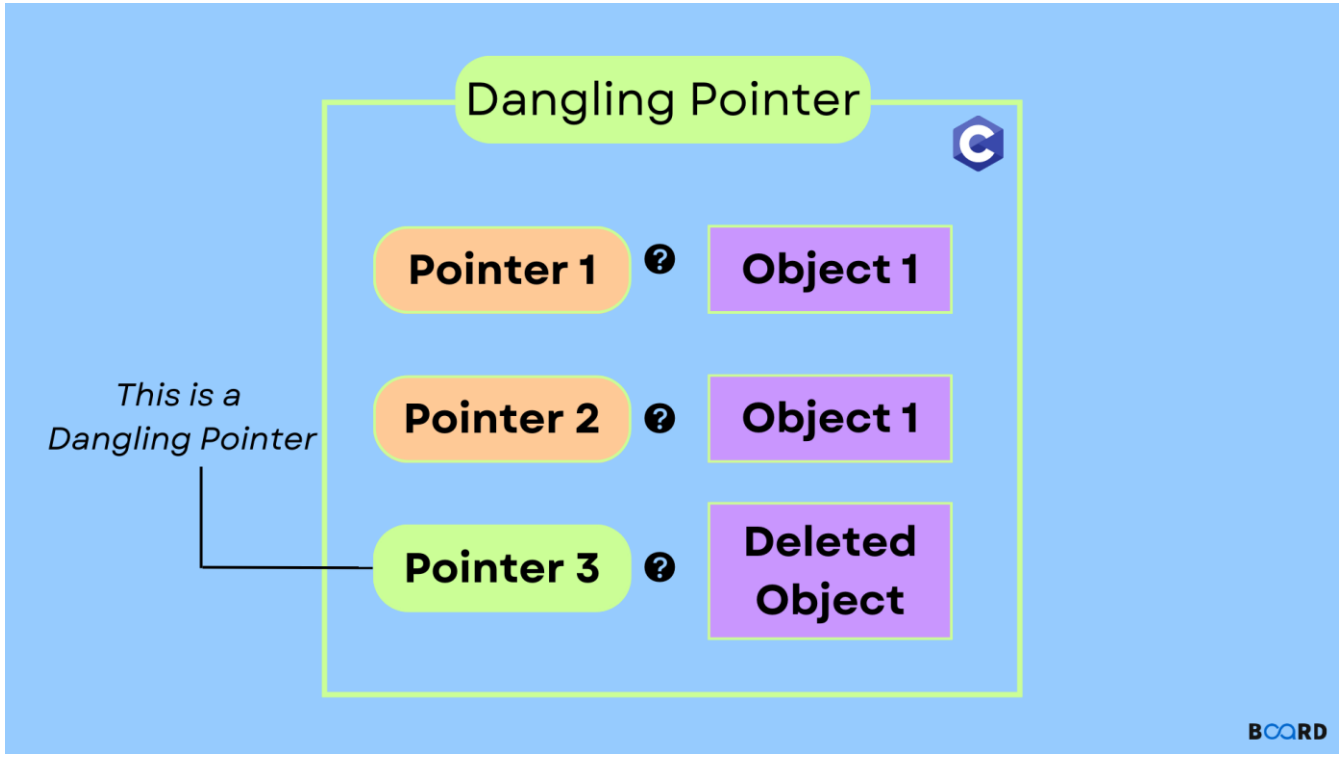
- Dangling pointers cause **undefined behavior** because they point to memory that is **no longer valid**

ptr

- Main problems:
  - **crashes**
    the **program may crash** when trying to access freed memory
  - **wrong values**
    the pointer may **read garbage** or unpredictable data

```
#include <iostream>
using namespace std;

int main() {
    int* ptr = new int;
    *ptr = 25;

    delete ptr;   // memory is released

    cout << *ptr << endl;   // dangling pointer: undefined behavior

    return 0;
}
```

Dangling pointers

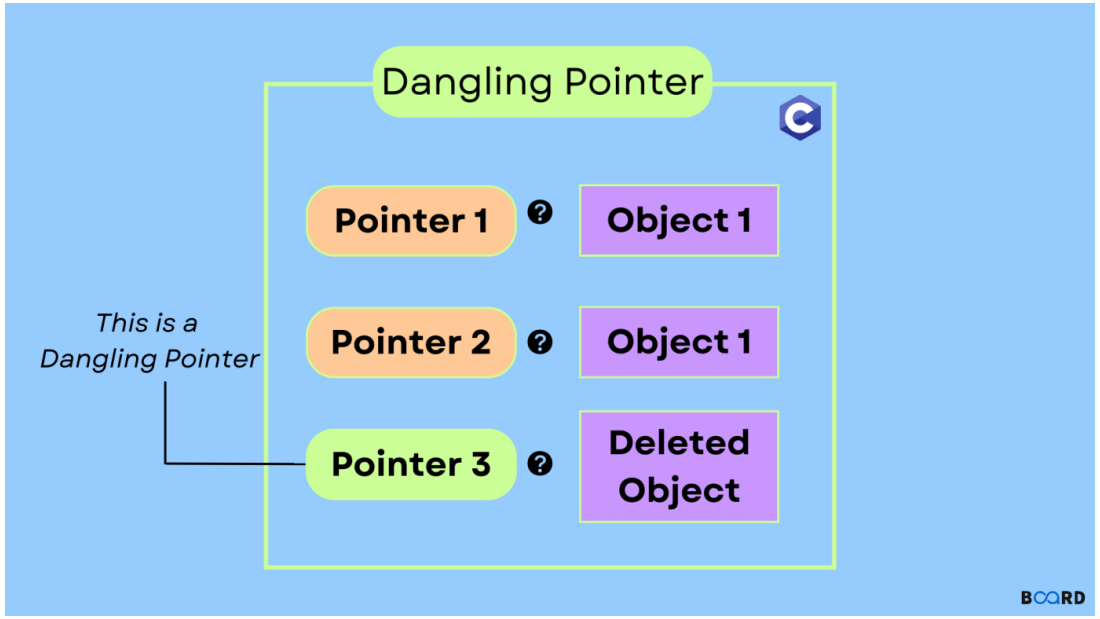


- To prevent a pointer from pointing to nothing (anymore), after a delete, it is advisable to:

```
delete [] p1;
p1=nullptr;
```

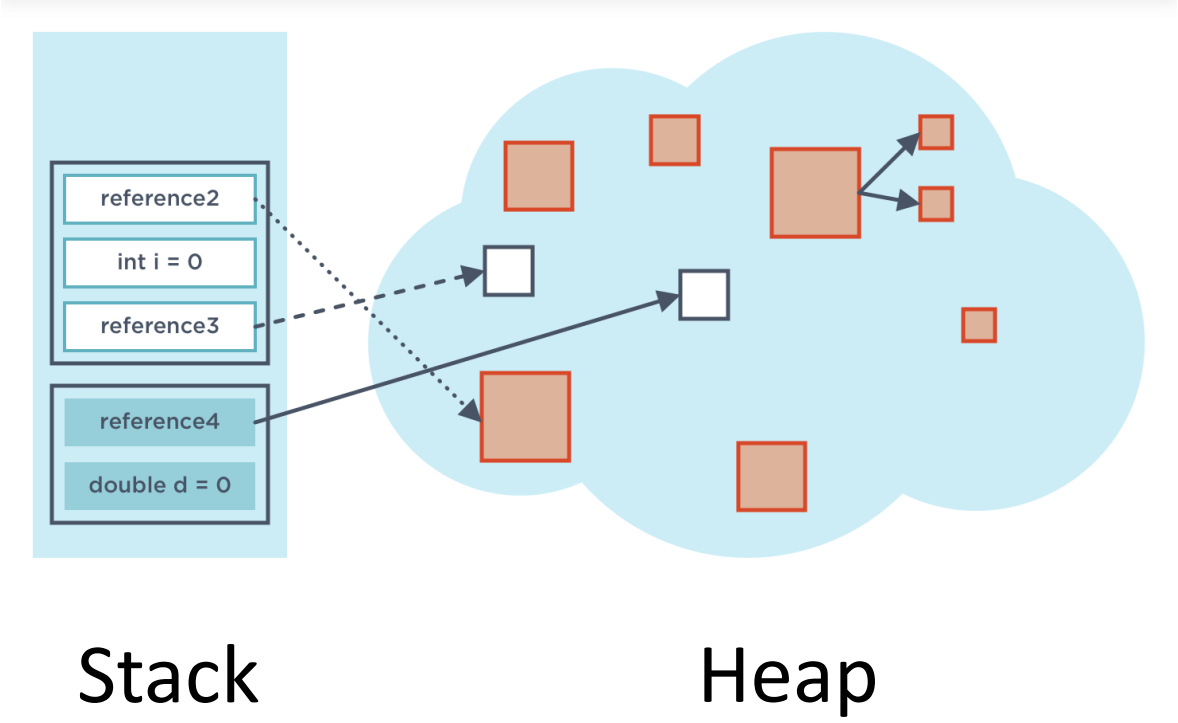
Exercise 1: Dangling pointers

- Write a simple C++ program that contains a dangling pointer
- Try to print the value associated with a dangling pointer



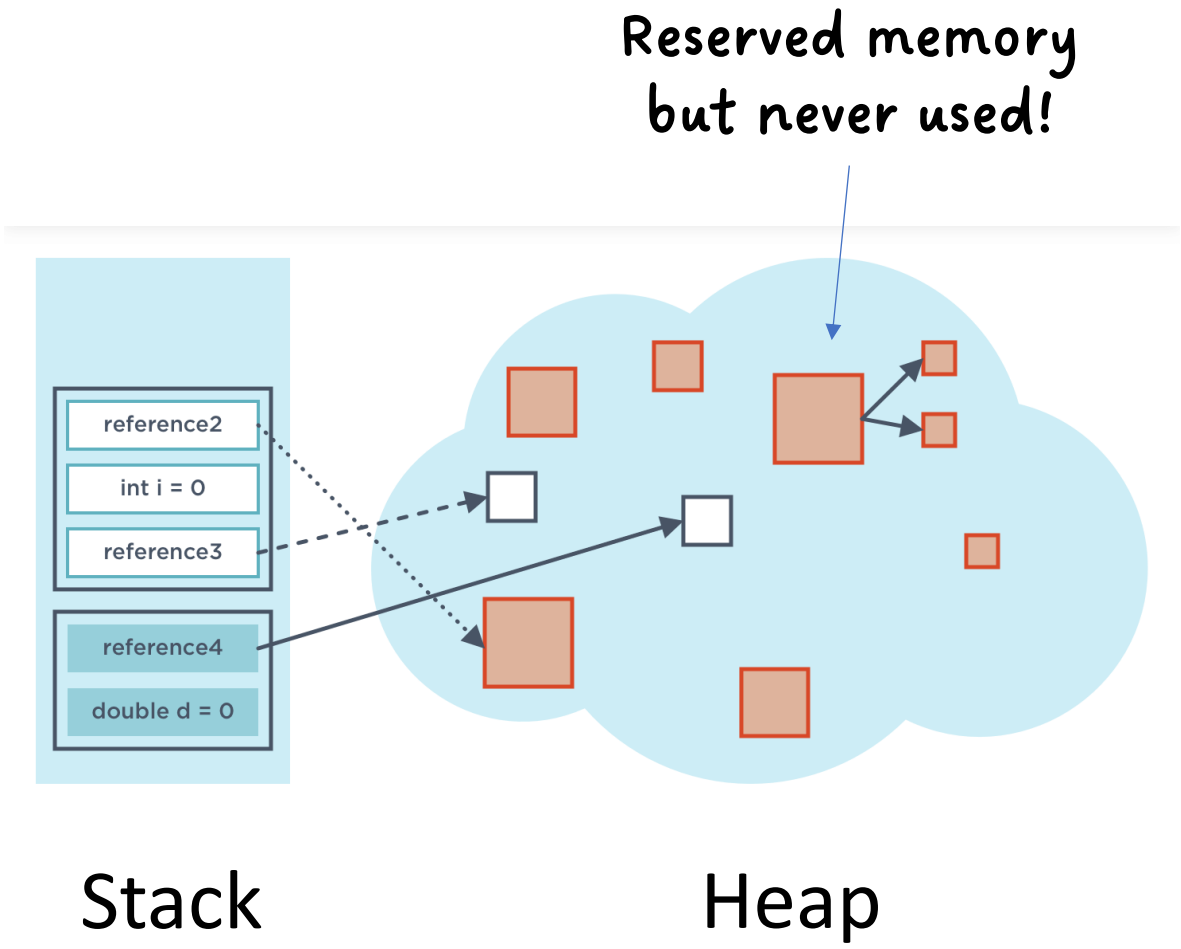
Memory Leak

- The **new** operator dynamically allocates memory
- When the memory is no longer needed, it should be released using **delete**
- If this is not done, the program **keeps memory that it no longer used**, causing a memory leak
- A memory leak is the loss of available memory caused by failing to deallocate dynamically allocated memory



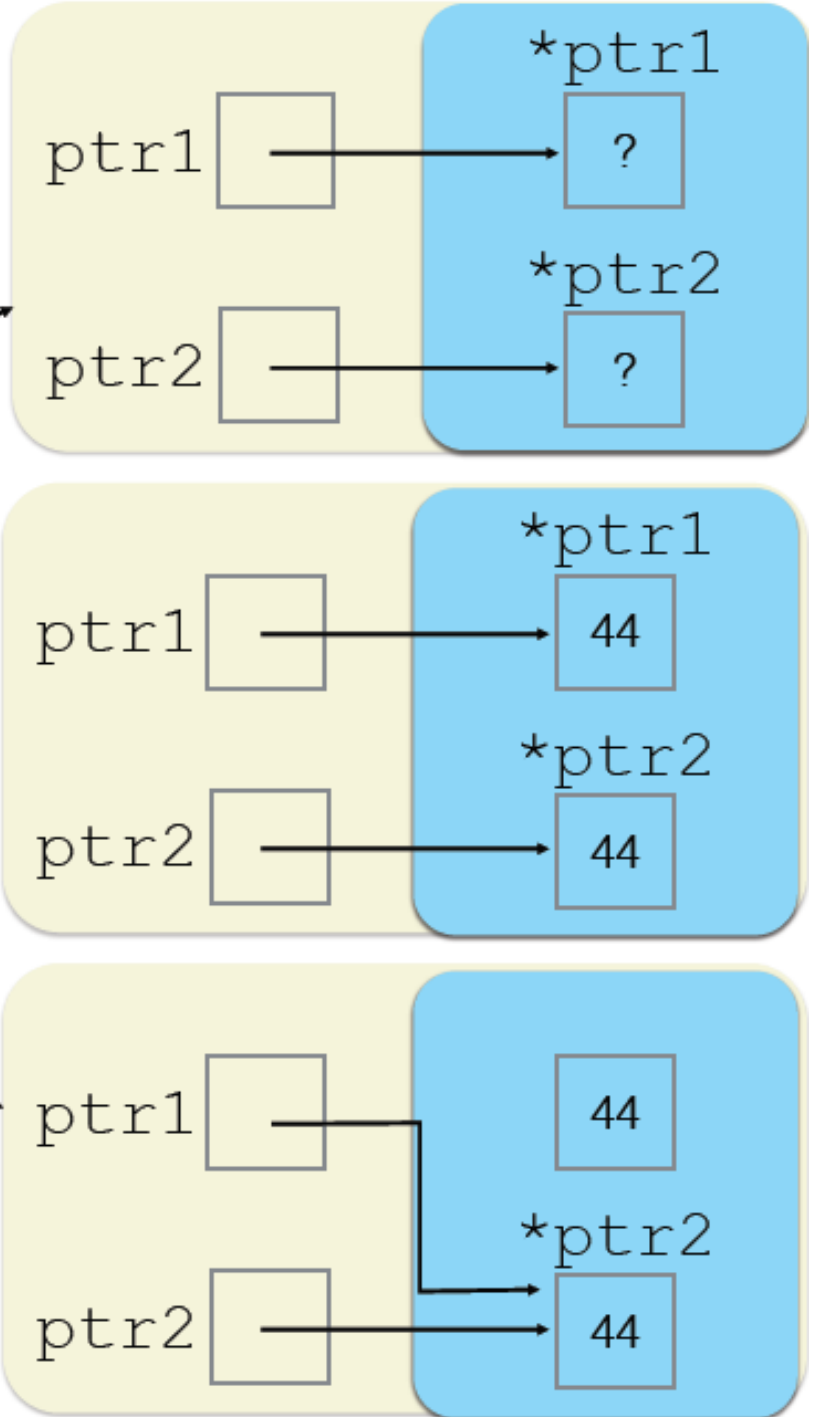
Memory Leak

- The **new** operator dynamically allocates memory
- When the memory is no longer needed, it should be released using **delete**
- If this is not done, the program **keeps memory that it no longer used**, causing a memory leak
- A memory leak is the loss of available memory caused by failing to deallocate dynamically allocated memory



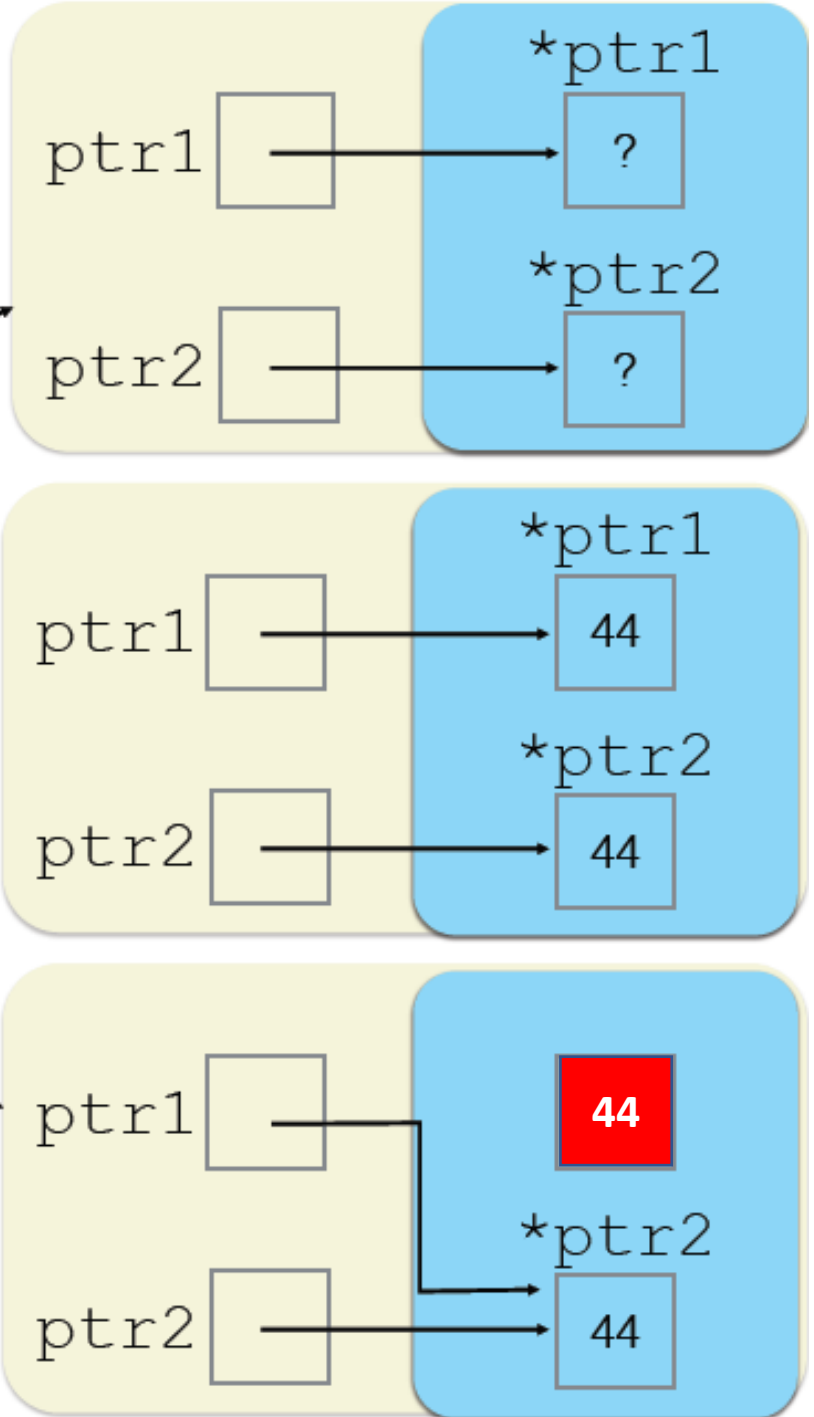
Memory Leak example

```
int *ptr1 = new int;
int *ptr2 = new int;

*ptr2 = 44;
*ptr1 = *ptr2;

ptr1 = ptr2;

delete ptr2;
ptr1=nullptr;
ptr2=nullptr;
```



Memory Leak example

```
int *ptr1 = new int;
int *ptr2 = new int;

*ptr2 = 44;
*ptr1 = *ptr2;

ptr1 = ptr2;

delete ptr2;
ptr1=nullptr;
ptr2=nullptr;
```



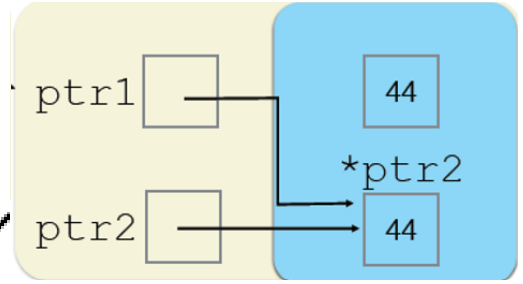
We forgot to delete ptr1, so that memory cell remains allocated

Memory Leak example and dangling pointer

```
int *ptr1 = new int;
int *ptr2 = new int;


*ptr2 = 44;
*ptr1 = *ptr2;


ptr1 = ptr2;


delete ptr2;
ptr2 = nullptr;
```

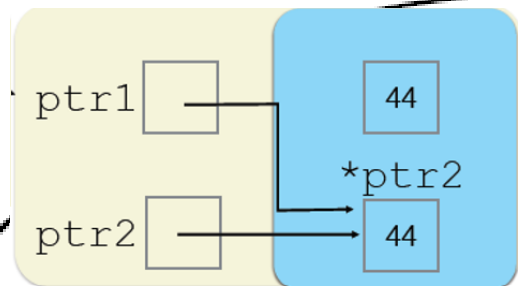
Memory Leak example and dangling pointer

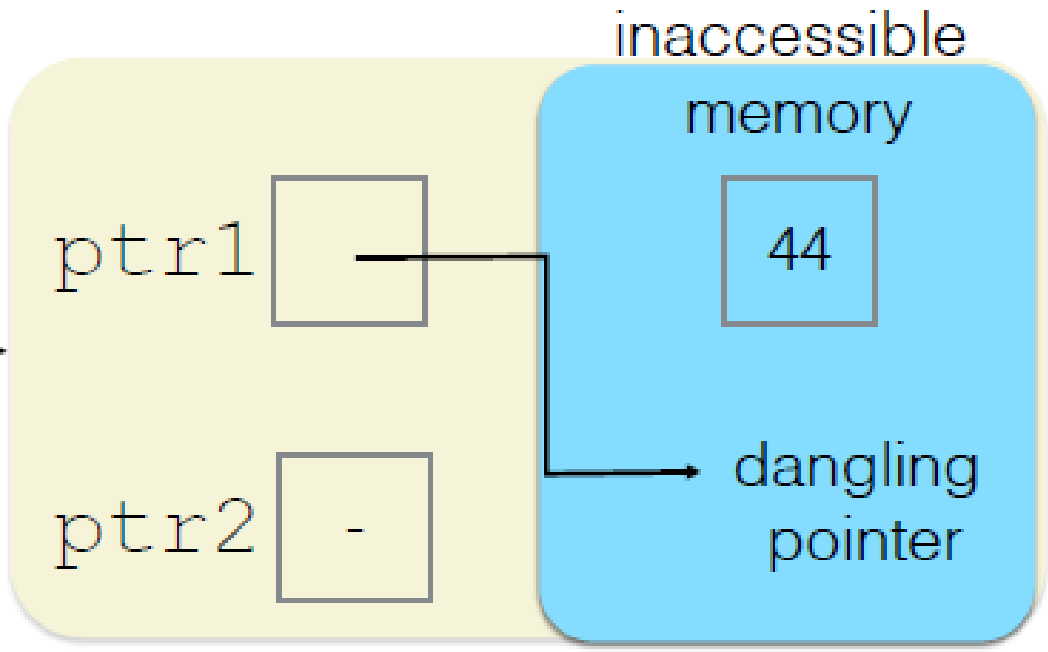
```
int *ptr1 = new int;
int *ptr2 = new int;


*ptr2 = 44;
*ptr1 = *ptr2;


ptr1 = ptr2;


delete ptr2;
ptr2 = nullptr;
```



Memory Leak example and dangling pointer

```
int *ptr1 = new int;
int *ptr2 = new int;


*ptr2 = 44;
*ptr1 = *ptr2;


ptr1 = ptr2;


delete ptr2;
ptr2 = nullptr;
```

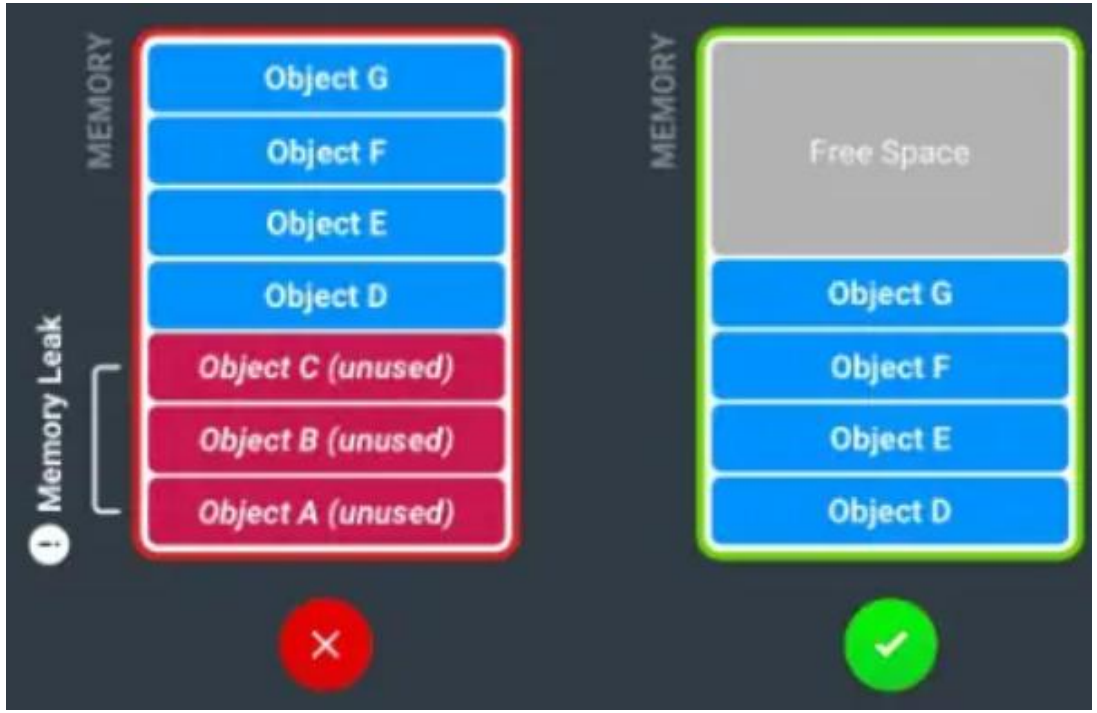




Memory Leak problems

- In large programs, memory leaks can build up over time, use more and more memory, **slow the program** down, and may eventually cause it to **crash**

- Tools like **Valgrind** or **Visual Studio's built-in memory profiling tools** can help detect and debug memory leaks
  - Advanced tools!
  - Not part of this course



All available memory is consumed!

Simple Strategy to Detect a Memory Leak

- Use a **counter** to track dynamic memory operations
- Every time new is called, the counter **increases** (**+1**)
- Every time delete is called, the counter **decreases** (**-1**)
- At the end of the program:
  - if the counter is 0, all allocated memory was released
  - if the counter is not 0, some memory was not released

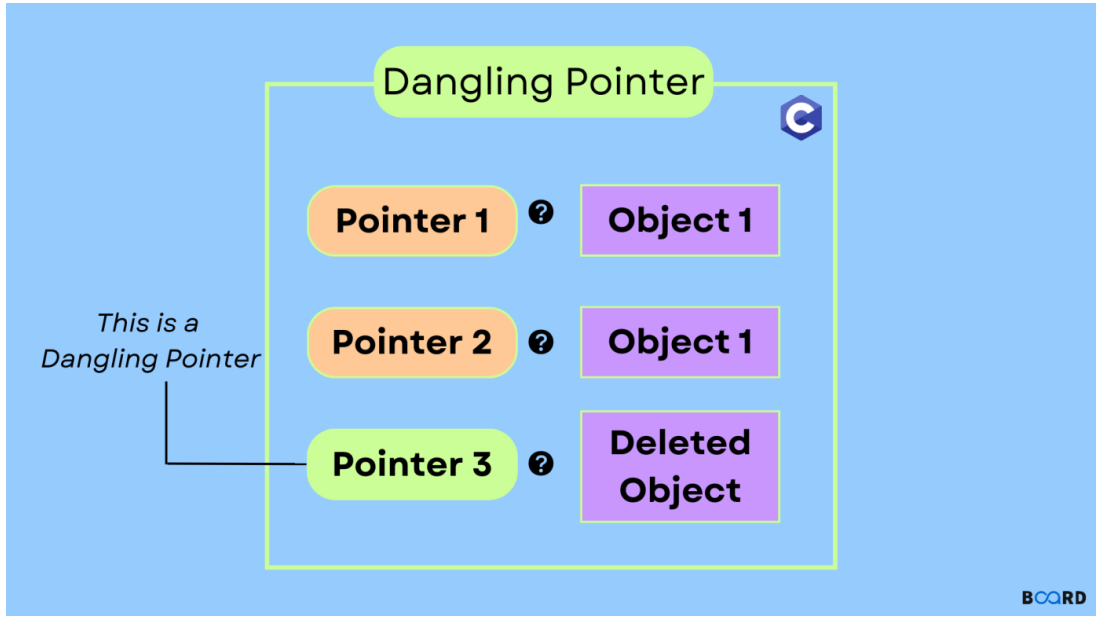
- **Takeaway:**
  If the final counter value is not zero, the program may contain a **memory leak**

```cpp
#include <iostream>
using namespace std;

int counter = 0;      // Global counter to track active memory allocations

void* operator new(size_t size) {   // Redefines the new operator
    counter++;                      // Increases the counter when memory is allocated
    return malloc(size);            // Allocates raw memory and returns its address
}

void operator delete(void* ptr) noexcept {   // Redefines the delete operator
    counter--;                               // Decreases the counter when memory is released
    free(ptr);                               // Frees the allocated memory
}

int main() {
    int* p1 = new int;        // Allocates memory for one integer
    int* p2 = new int;        // Allocates memory for another integer

    delete p1;                // Frees the memory pointed to by p1
    // delete p2;            // This line is commented out, so p2 is not freed

    cout << "Allocations not freed: " << counter << endl;   // Prints how many allocations were not released

    return 0;
}
```

This code uses advanced C++ concepts, such as operator overloading, which we will cover in the final lectures

Exercise 2: Memory leak

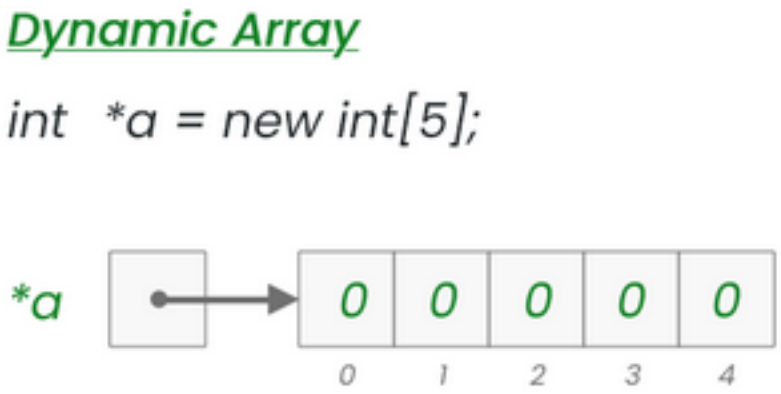
- Write a simple C++ program that contains Memory leaks
- Try to execute the program given in the previous slide



Dynamic arrays

Dynamic array

- A **dynamic array** in C++ is an array whose **size can be determined at runtime**, as opposed to a static array, where the size is fixed at compile-time
- Dynamic arrays are **useful when the size of the array is not known in advance** or may change during the execution of the program
- In C++, dynamic arrays are created **using pointers** and the new keyword for memory allocation and delete or delete[] for deallocation



Dynamic arrays vs static arrays

```
int a[10];
int *p;
p = new int[10];
```

Static array

Dinamic array

a	p
resides in the stack	resides in the heap
fixed size **Compile time**	variable size **Run time**
static binding between name and array (*a* is a **constant pointer**)	dynamic binding between name and array

Dynamic arrays vs static arrays

```
int a[10];
```

Static array

```
int *p;
p = new int[10];
```

Dinamic array

a	p
resides in the stack	resides in the heap
fixed size **Compile time**	variable size **Run time**
static binding between name and array (*a* is a **constant pointer**)	dynamic binding between name and array

Dynamic arrays vs static arrays

```
int a[10];
```

Static array

```
int *p;
p = new int[10];
```

Dinamic array

a	p
resides in the stack	resides in the heap
fixed size **Compile time**	variable size **Run time**
static binding between name and array (*a* is a **constant pointer**)	dynamic binding between name and array

p is a real pointer variable, so it can later point somewhere else

Static Arrays vs Variable-Size Dynamic Arrays

- Static arrays require a **size known before execution**, while dynamic arrays can be created with a **size determined at runtime**

```cpp
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;

    int arr[n];   // compile-time error in standard C++
    return 0;
}
```

Static array

Static Arrays vs Variable-Size Dynamic Arrays

- Static arrays require a **size known before execution**, while dynamic arrays can be created with a **size determined at runtime**

```cpp
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;

    int arr[n];   // compile-time error in standard C++
    return 0;
}
```

Static array

Dynamic array

```cpp
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;

    int* arr = new int[n];   // size decided at runtime

    delete[] arr;
    return 0;
}
```

Dynamic arrays and access to elements

- **Access to elements can be performed as with static arrays** (including pointer arithmetic)

```
int *p;
p = new int[5];
for (int i=0;i<5;++i)
        p[i]=i*i;
```

```
int *p;
p = new int[5];
for (int i=0;i<5;++i)
        *(p+i)=i*i;
```

Dynamic arrays and access to elements

- **Access to elements can be performed as with static arrays** (including pointer arithmetic)

```
int *p;
p = new int[5];
for (int i=0;i<5;++i)
        p[i]=i*i;
```

```
int *p;
p = new int[5];
for (int i=0;i<5;++i)
        *(p+i)=i*i;
```

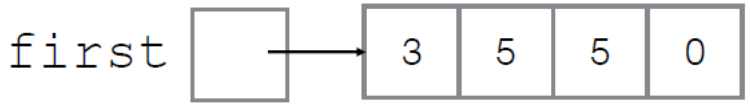
- Just like in the static case, **accessing memory outside the range** (out of range) can result in one of two scenarios:
  - If the cell is outside the memory allocated by the heap, the **program aborts**
  - If the cell is within the allocated area, the program continues but produces **unpredictable results**

Shallow copy and deep copy

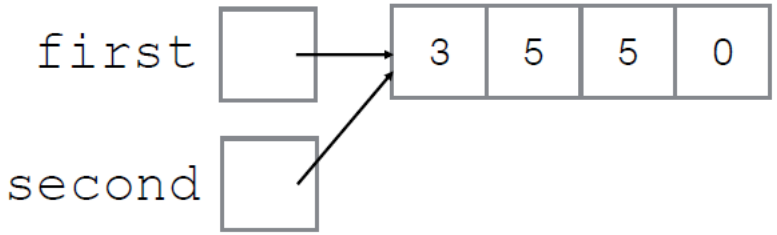
- Consider the following statements

```
int *first;
int *second;
first =  new int[10];
```

- Insert some elements in the ‘first’ array



- **Shallow copy**:
  - second = first



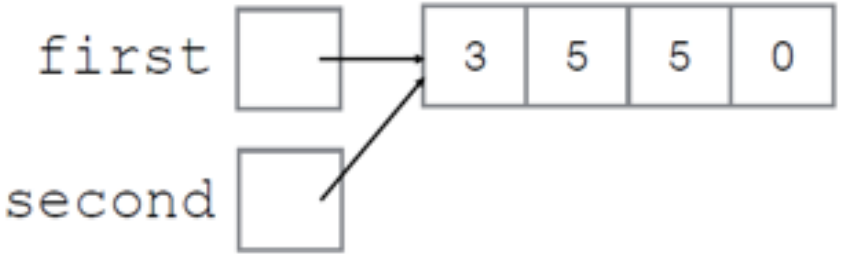
Shallow copy and deep copy

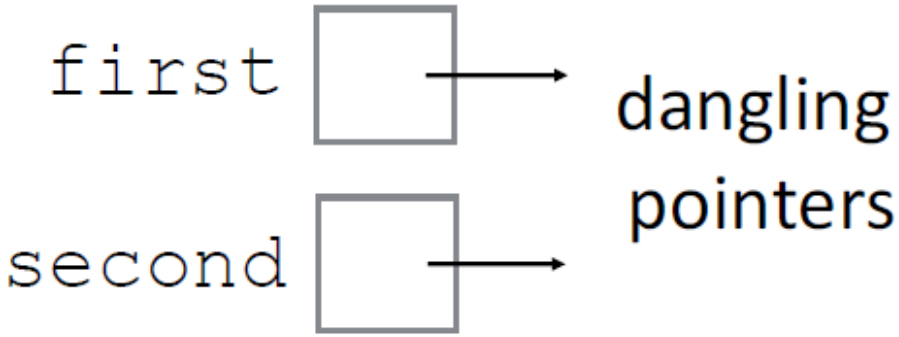
- what happens if:

```
delete [] second;
?
```

Shallow copy and deep copy

- what happens if: `delete [] second;`
  `?`

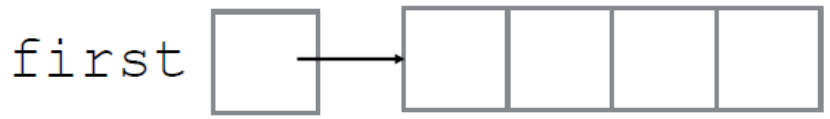




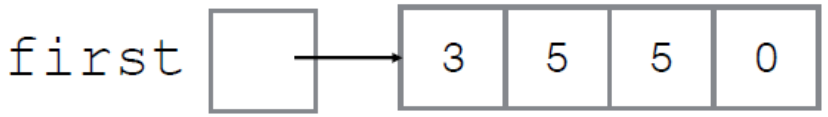
Shallow copy and deep copy

- Consider the following statements

```
int *first;
int *second;
first =  new int[10];
```

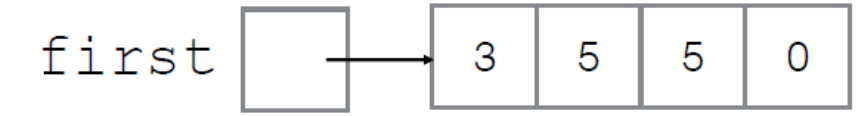


- Insert some elements in the ‘first’ array



- **Deep copy:**

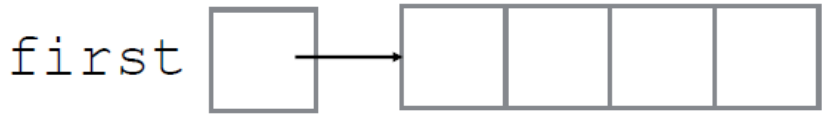
```
second = new int[10];
for (int i=0;i<10;i++)
 second[i]=first[i];
```



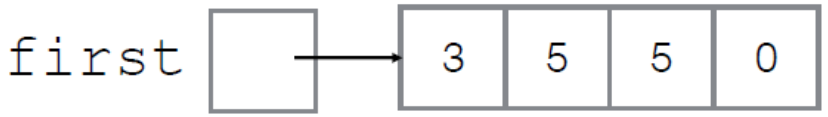
Shallow copy and deep copy

- Consider the following statements

```
int *first;
int *second;
first =  new int[10];
```



- Insert some elements in the 'first' array



- **Deep copy:**

```
second = new int[10];
for (int i=0;i<10;i++)
 second[i]=first[i];
```

Now `delete [] second;` only deletes the data associated with second

Resizing a Dynamic Array

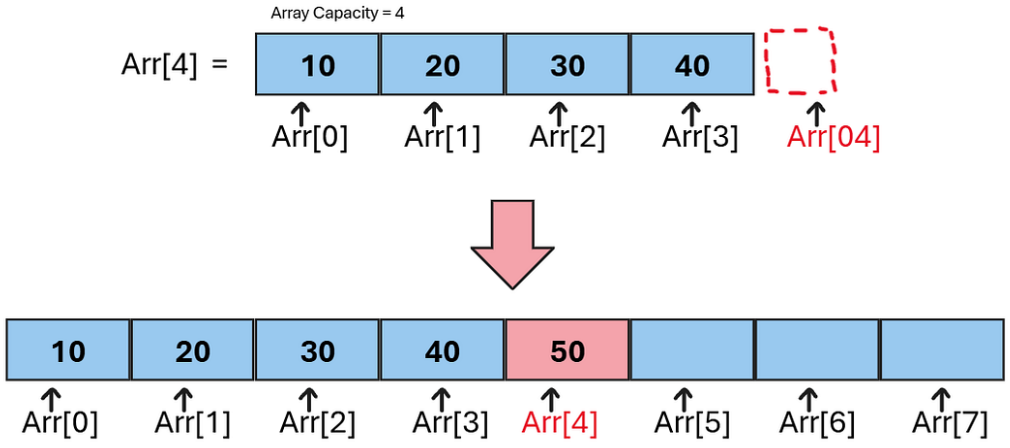
- A dynamic array **cannot grow by itself**
- For example:
  - int* p = new int[4];
- Here, the array has space for exactly **4 elements**. If you later need to store a 5th element, the existing array cannot simply expand in place

Solution:

Resize: if the current dynamic array no longer has enough space, we must create another **bigger array** and move the data into it

Basic idea:

1. Allocate a **new array** with a larger size
2. Copy the elements from the old array into the new one (**deep copy**)
3. Delete the old array
4. Make the pointer refer to the new array



We create a bigger array

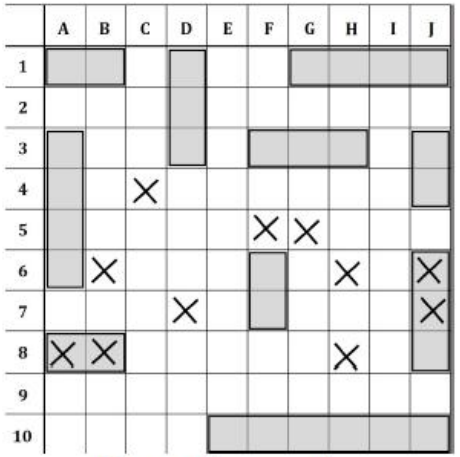
```cpp
#include <iostream>
using namespace std;

int main() {
    int size = 5;                   // it is no longer a const
    int* a = new int[size];         // allocation on the heap
    int n = 0;                      // number of inserted elements
    int value;                      // temporary variable for input

    // --- Read values into the array until 0 is entered
    while (cin >> value && value != 0) {
        if (n >= size) {                            // when the array is full
            size = size * 2;                        // double the value of size
            int* temp = new int[size];              // create a temporary array twice as large
            for (int i = 0; i < n; ++i) {
                temp[i] = a[i];                     // deep copy
            }
            delete[] a;                             // free the old (smaller) array
            a = temp;                               // update the pointer
        }

        a[n] = value;                               // store the new value in the array
        n++;                                        // increase the number of inserted elements
    }

    // --- Print what was inserted (it may be less than size!)
    for (int i = 0; i < n; ++i)
        cout << a[i] << endl;
    delete[] a;                                     // free the dynamic array before the program ends
    return 0;
}
```

Dynamic ad static arrays

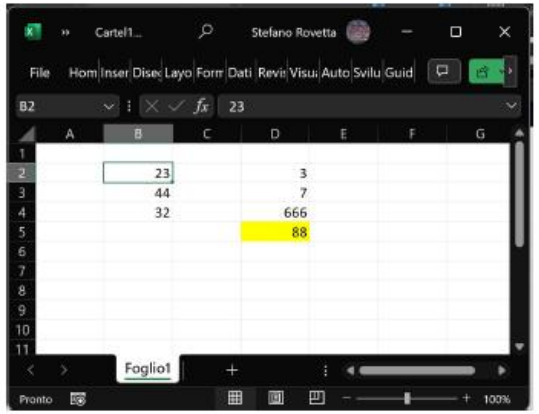
```
#include <iostream>
using namespace std;

const int SIZE = 10;   // fixed size used for both arrays

void print_array(int *a, int size) {   // prints all elements of the array
    for (int i = 0; i < size; ++i)
        std::cout << *(a + i) << " ";  // prints the value at position i
    std::cout << std::endl;            // moves to the next line
}

void init_array(int *a, int size, int K) {   // initializes all elements to K
    for (int i = 0; i < size; ++i)
        *(a + i) = K;                      // stores K in each array position
}

int main() {
    int *p1;                        // pointer to a dynamic array
    p1 = new int[SIZE];             // dynamic array allocated on the heap
    int v[SIZE];                    // static array allocated on the stack

    init_array(v, SIZE, 1);         // fills the static array with 1
    print_array(v, SIZE);           // prints the static array

    init_array(p1, SIZE, 3);        // fills the dynamic array with 3
    print_array(p1, SIZE);          // prints the dynamic array

    delete[] p1;                    // releases the dynamic array
    return 0;                       // ends the program
}
```

They are equivalent when used as parameters in a Function

Bi-dimensional arrays



Battaglia navale



Foglio di calcolo

Pixel (immagine)

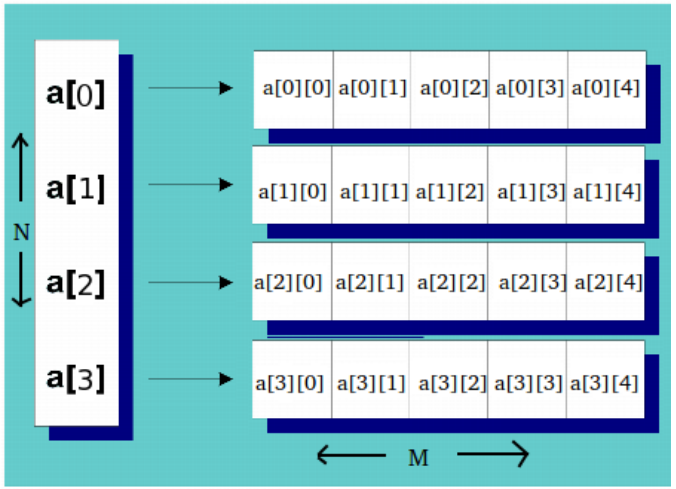
- Characterized by two indices: a 'number of rows' and a 'number of columns

Already introduced in Lecture 14

(Static) Bi-dimensional arrays

- They are arrays of arrays – Arrays whose elements are themselves arrays

```
A[i][j] = element in row i, column j
A[i]    = the entire row i (an array)
```



- **Always allocated on the stack!!!**

(Dynamic) Bi-dimensional arrays 1

- **Always allocated on the heap!**

```
int** m = new int*[rows];
```

First step in creating a **dynamic 2D array**

(Dynamic) Bi-dimensional arrays 1

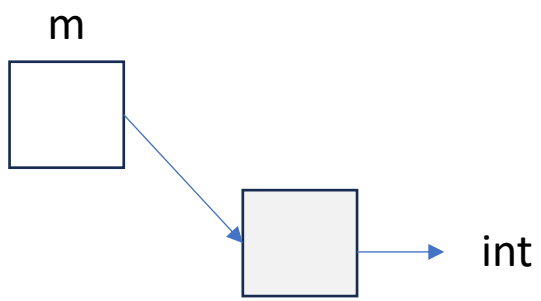
- **Always allocated on the heap!**

```
int** m = new int*[rows];
```

First step in creating a **dynamic 2D array**

- **1. int** m**

  This means:
  - m is a **pointer to a pointer to int**
  - in practice, m will store the address of an array of pointers
  - each of those pointers will later point to one row



(Dynamic) Bi-dimensional arrays 1

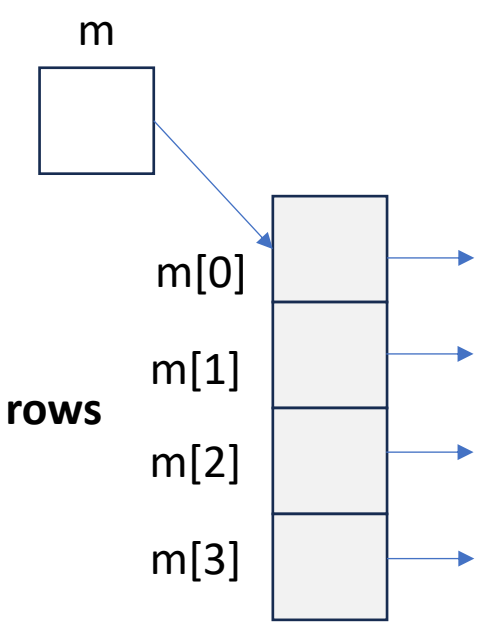
- **Always allocated on the heap!**

```
int** m = new int*[rows];
```

First step in creating a **dynamic 2D array**

- **1. int** m**

  This means:
  - m is a **pointer to a pointer to int**
  - in practice, m will store the address of an array of pointers
  - each of those pointers will later point to one row



- **2. new int*[rows]**
  - This allocates memory for:
    - an array with rows elements
    - each element is of type int*
  - So this line creates: an array of **row pointers**

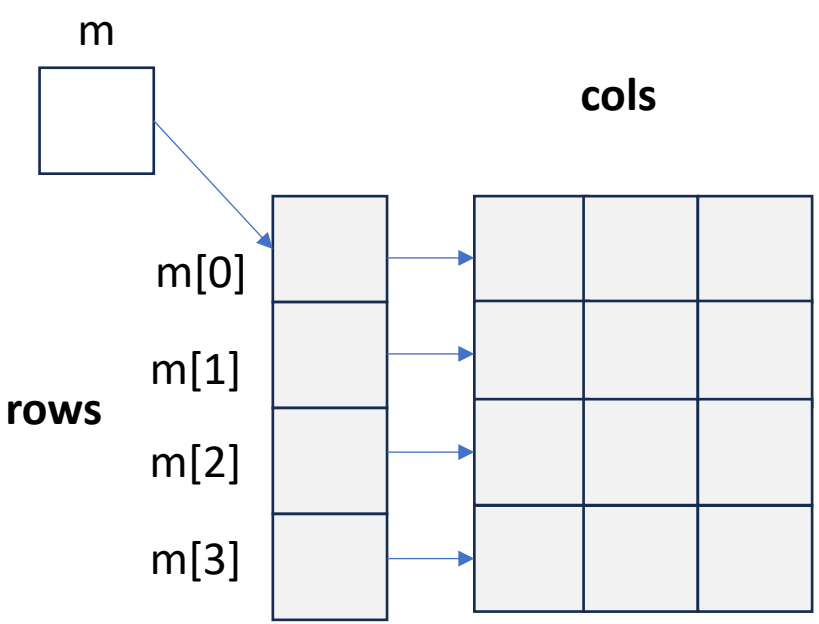
(Dynamic) Bi-dimensional arrays 2

- **3. What happens next**

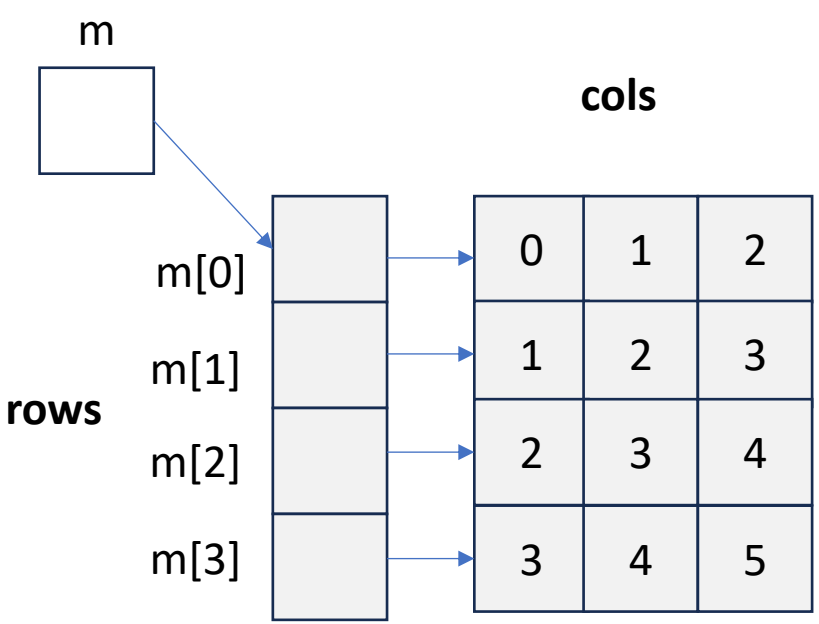
Usually after that, you allocate each row:

```
for (int i = 0; i < rows; i++) {
    m[i] = new int[cols];
}
```

Now each m[i] points to an array of cols integers



```cpp
#include <iostream>
using namespace std;

int main() {
    int rows = 4;
    int cols = 3;

    int** m = new int*[rows];          // allocate rows
    for (int i = 0; i < rows; i++) {
        m[i] = new int[cols];          // allocate columns for each row
    }

    // example initialization
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            m[i][j] = i + j;
        }
    }

   // print the matrix
   for (int i = 0; i < rows; i++) {
       for (int j = 0; j < cols; j++) {
           cout << m[i][j] << " ";
       }
       cout << endl;
   }

   // deallocation
   for (int i = 0; i < rows; i++) {
       delete[] m[i];
   }
   delete[] m;

   return 0;
}
```



Exercise 3: Dynamic String Array Input and Output

- Write a C++ program that takes ‘n’ strings from the user and **dynamically allocates** an array to store those strings
- Show all the entered strings along with their corresponding indices

- Example run:

```
Enter the number of strings: 2
String 1: dynamic allocation
String 2: memory leaks

Output:
[0]: dynamic allocation
[1]: memory leaks
```